Microsoft SQL Server Receiver

The `sqlserver` receiver grabs metrics/logs about a Microsoft SQL Server instance. The receiver works by either using the
Windows Performance Counters, or by directly connecting to the instance and querying it. Windows Performance Counters
are only available when running on Windows.

Status	
Stability	development: logs
	beta: metrics
Distributions	contrib
Issues	open 7 closed 86
Code coverage	codecov 82%
Code Owners	@sincejune, @crobert-1, @sv-splunk, @ebrdarSplunk, @XSAM, @akshays-19, @splunk-shanu | Seeking more code owners!
Emeritus	@StefanKurek

Required Permissions

Windows Performance Counters

Make sure to run the collector as administrator in order to collect all performance counters for metrics.

Direct Connection

When configured to directly connect to the SQL Server instance, the user must have the following permissions:

1. At least one of the following permissions:

- `CREATE DATABASE`
- `ALTER ANY DATABASE`
- `VIEW ANY DATABASE`

2. Permission to view server state:

    - SQL Server pre-2022: `VIEW SERVER STATE`
    - SQL Server 2022 and later: `VIEW SERVER PERFORMANCE STATE`

3. To collect the per-index physical stats metrics (`sqlserver.index.*`), the following two permissions are also required:

- `CONNECT ANY DATABASE` — the receiver enters each user database to read its index physical stats; this grant lets the login connect to all current and future databases.
- `VIEW ANY DEFINITION` — makes the index, object, and schema catalog views visible in every database so the physical stats query can resolve index metadata.

4. To collect the `db.server.top_procedure` event, `VIEW ANY DEFINITION` is also required. The query resolves each procedure's schema and name with `OBJECT_SCHEMA_NAME` and `OBJECT_NAME` across databases, and both return `NULL` without metadata visibility in the target database. Rows that cannot be resolved are skipped, so a login missing this grant sees procedures silently absent rather than an error. Procedures in the system databases (`master`, `tempdb`, `model`, `msdb`) are never reported.

> [!NOTE]
> `db.server.top_procedure` requires SQL Server 2017 CU3 or later, where `sys.dm_exec_procedure_stats.total_spills` became available. On older builds the event
> reports nothing and logs an error; the other events are unaffected.

Configuration

The following is a generic configuration that can be used for the default logs and metrics scraped by the SQL Server receiver. A basic explanation on some of the fields has also been provided. For more information, please reference the following section.

```
sqlserver:
  collection_interval: 10s                    # interval for overall
collection
  instance_name: CustomInstance
  username: myusername
  password: mypassword
  server: sqlserver.address
  port: 1433
  events:
    db.server.query_sample:
      enabled: true
    db.server.top_query:
      enabled: true
    db.server.top_procedure:
      enabled: true
    db.server.query_plan:
      enabled: true
  top_query_collection:                       # this collection exports
the most expensive queries as logs
    lookback_time: 60s                        # which time window
should we look for the top queries
    max_query_sample_count: 1000              # maximum number query we
store in cache for top queries.
    top_query_count: 250                      # The maximum number of
active queries to report in a single run.
```

```
    collection_interval: 60s                  # collection interval for
top query collection specifically
  query_sample_collection:                    # this collection exports
the currently (relate to the query time) executing queries as logs
    max_rows_per_query: 100                   # the maximum number of
samples to return for one single query.
  top_procedure_collection:                   # this collection exports
aggregated stored procedure statistics as logs
    max_procedure_sample_count: 1000          # maximum number of
procedures to consider as candidates in a single run.
    top_procedure_count: 250                  # The maximum number of
procedures to report in a single run.
    collection_interval: 60s                  # collection interval for
top procedure collection specifically
```

The following settings are optional:

- `collection_interval` (default = `10s`): The interval at which metrics should be emitted by this receiver.
- `instance_name` (optional): The instance name identifies the specific SQL Server instance being monitored.
  If unspecified, metrics will be scraped from all instances. If configured, the `computer_name` must also be set
  when running on Windows.
- `top_procedure_collection` (optional): Tunes the `db.server.top_procedure` event.
  - `max_procedure_sample_count` (default = `1000`, max `10000`): How many procedures to read from
    `sys.dm_exec_procedure_stats` as candidates each run.
  - `top_procedure_count` (default = `250`): How many of those candidates to report, chosen by the
    elapsed time each procedure accrued since the previous run. Must not exceed `max_procedure_sample_count`.
  - `collection_interval` (default = `60s`): The interval at which top procedures should be emitted by this receiver, independently of the global `collection_interval`. As with `top_query_collection.collection_interval`, this only guarantees the event is collected at most
    once per interval: the scraper still runs on the global interval and skips the collection until this much time has passed since the last one, so a value below the global interval has no effect. It also sets the window the reported deltas cover, so raising it rolls the counters up over a longer period. Candidates are also bounded to procedures that actually ran within that
    same window (derived automatically from the time since the last successful scrape, with no separate setting to tune), so a procedure is only sampled if it executed recently, not merely because it has a large lifetime total.

Direct connection options (optional, but all must be specified to enable):

- `username`: The username used to connect to the SQL Server instance.

- `password`: The password used to connect to the SQL Server instance.
- `server`: IP Address or hostname of SQL Server instance to connect to.
- `port`: Port of the SQL Server instance to connect to.

For extension-provided credentials, configure `db_auth` with the component ID of an extension implementing `dbauth.Provider` instead of `password`. The provider is resolved from the Collector's `extensions` map and may refresh credentials for each new database connection. `db_auth` and `password` are mutually exclusive; `username` may still be supplied when the provider uses it as input. The `azure_auth` extension can provide Azure SQL tokens this way. Set its `scopes` to the Azure SQL resource scope for the target cloud (for example, `https://database.windows.net/.default` for Azure Public); the default is Azure Public SQL.

For finer control over the direct connection use the `datasource`, a.k.a. the "connection string", instead. Note: it can't be used in conjunction with the `username`, `password`, `server` and `port` options.

When a direct connection is used, all scrapers created for a signal share a single database connection pool
instead of each opening its own. The pool is scoped per receiver instance: if this receiver is used in both a
metrics and a logs pipeline, the metrics and logs receivers each own a separate pool. The pool can be tuned
with the `connection_pool` options (all optional):

- `max_open` (default = number of scrapers): The maximum number of open connections to the database. `0` means unlimited.
- `max_idle` (default = number of scrapers): The maximum number of idle connections kept in the pool.
- `max_lifetime` (optional, example = `5m`, default = unset): The maximum amount of time a connection may be reused. `0` means connections are reused forever.
- `max_idle_time` (optional, example = `1m`, default = unset): The maximum amount of time a connection may be idle before being closed. `0` means idle connections are not closed due to idle time.

The defaults are derived from the number of enabled scrapers so that every scraper can query concurrently
while keeping the total number of connections bounded. Most deployments do not need to set these; tune them
only when connecting to an instance with strict connection limits or a large number of enabled scrapers.

Windows-specific options:

- `computer_name` (optional): The computer name identifies the SQL Server name or IP address of the computer being monitored.
  If specified, `instance_name` is also required to be defined. This option is ignored in non-Windows environments.

Top-Query collection specific options (only useful when top-query collection are enabled):

- `lookback_time` (optional, example = `60s`, default = `2 * collection_interval`): The time window (in second) in which to query for top queries.
    - Queries that were finished execution outside the lookback window are not included in the collection. Increasing the lookback window (in seconds) will be useful for capturing long-running queries.
- `max_query_sample_count` (optional, example = `5000`, default = `1000`): The maximum number of records to fetch in a single run.
- `top_query_count`: (optional, example = `100`, default = `250`): The maximum number of active queries to report (to the next consumer) in a single run.
- `collection_interval`: (optional, default = `60s`): The interval at which top queries should be emitted by this receiver.
    - This value can only guarantee that the top queries are collected at most once in this interval.
        - For instance, you have global `collection_interval` as `10s` and `top_query_collection.collection_interval` as `60s`.
            - In this case, the default receiver scraper will still try to run in every 10 seconds.
            - However, the top queries collection will only run after 60 seconds have passed since the last collection.
        - For instance, you have global `collection_interval` as `10s` and `top_query_collection.collection_interval` as `5s`.
            - In this case, `top_query_collection.collection_internal` will make no effects to the collection

By default, `db.server.top_query` carries the query's execution plan in its `sqlserver.query_plan` attribute. Execution plans can be large, so an oversized plan can push its record past a transport/buffer limit and take the lightweight query statistics down with it. Enabling the `db.server.query_plan` event (disabled by default, like the other events above) moves the plan onto its own record: `db.server.top_query` is then emitted without `sqlserver.query_plan`, and the plan
is reported on `db.server.query_plan` instead, joined back via the composite key `sqlserver.query_hash` + `sqlserver.query_plan_hash` (the plan hash alone can collide across distinct queries with a similar shape). Leaving `db.server.query_plan` disabled preserves the previous behavior exactly.

`db.server.query_plan` is sourced from the same query as `db.server.top_query` and only splits the
plan out of it, so it collects nothing unless `db.server.top_query` is enabled too.

Query sample collection related options (only useful when query sample is enabled)

- `max_rows_per_query`: (optional, default = `100`) use this to limit rows returned by the sampling query.
  Example:

```
receivers:
  sqlserver:
    collection_interval: 10s
  sqlserver/1:
    collection_interval: 5s
    username: sa
    password: securepassword
    server: 0.0.0.0
    port: 1433
```

When a named instance is used on Windows, a computer name and an instance name must be specified.
Example with named instance:

```
receivers:
  sqlserver:
    collection_interval: 10s
    computer_name: CustomServer
    instance_name: CustomInstance
    resource_attributes:
      sqlserver.computer.name:
        enabled: true
      sqlserver.instance.name:
        enabled: true
```

The full list of settings exposed for this receiver are documented in config.go with detailed sample configurations in testdata/config.yaml.

Top query collection enabled:

```
receivers:
  sqlserver:
    collection_interval: 5s
    username: sa
    password: securepassword
    server: 0.0.0.0
    port: 1433
    top_query_collection:
      lookback_time: 60s
      max_query_sample_count: 1000
      top_query_count: 200
    query_sample_collection:
      max_rows_per_query: 1450
```

Resource attributes

`server.address` and `server.port` identify the monitored SQL Server instance and are emitted by default.
When the receiver connects over loopback (for example `server: localhost` or `server: 127.0.0.1`),
`server.address` reports the host name of the machine running the collector, because the monitored instance is co-located with it and `localhost` would otherwise be shared by every monitored host.
`service.instance.id` resolves its host the same way, so it reports the collector host name rather than
`localhost` for a loopback target.

To stop emitting the server attributes, disable them individually:

```
sqlserver:
  resource_attributes:
    server.address:
      enabled: false
    server.port:
      enabled: false
```

`db.system.version` reports the SQL Server product version (e.g. `15.0.4261.1`). It is disabled by default and only available in direct connection mode. Enable it via:

```
sqlserver:
  resource_attributes:
    db.system.version:
      enabled: true
```

> **Note:** On Azure SQL Database and Azure SQL Managed Instance,
> `SERVERPROPERTY('ProductVersion')`
> returns a fixed legacy version string rather than the actual engine version.

Metrics

Details about the metrics produced by this receiver can be found in documentation.md

Logs

Details about the logs produced by this receiver can be found in logs-documentation.md

Known issues

`x509: negative serial number` with SQL Server in Docker

SQL Server docker users may run into an issue that the collector fails to parse certificate from server due to `x509: negative serial number`. That's because we adopted Go `1.23` starting from contrib `v0.121.0`:

> Before Go 1.23, ParseCertificate accepted certificates with negative serial numbers.
> This behavior can be restored by including "x509negativeserial=1" in the GODEBUG environment variable.

references:

1. https://pkg.go.dev/crypto/x509#ParseCertificate
2. https://github.com/microsoft/mssql-docker/issues/895

TLS handshake failures on Windows Server 2012 R2 and older

Direct connections to SQL Server hosted on Windows Server 2012 R2 or older may fail during the TLS handshake. See `TLS Handshake failed: cannot read handshake packet: EOF` in Troubleshooting.

Troubleshooting

`service.instance.id` is `unknown:1433`

In a rare case, the `service.instance.id` resource attribute is set to `unknown:1433`. This is because the receiver is unable to parse and compute the `service.instance.id` resource attribute.

You can file an issue that includes your configuration to help us investigate the issue.

`TLS Handshake failed: cannot read handshake packet: EOF`

This error has been reported for SQL Server on Windows Server 2012 R2 and older. It occurs even when `ForceEncryption` is set to `0`, because by default the driver still uses TLS to encrypt the login packet.

The root cause is unconfirmed. Setting the minimum TLS version (`tlsmin`) and re-enabling RSA key exchange (`tlsrsakex`) through `GODEBUG` did not resolve it. See #50774 for the investigation so far.

Options, in order of preference:

1. Upgrade the host to a newer Windows Server release.
2. Review the certificate and the TLS cipher suite configuration on the SQL Server host.
3. As a last resort, disable encryption for the connection by adding `encrypt=disable` to the `datasource`.

> [!WARNING]
> With the default setting only the login packet is encrypted, and `encrypt=disable` removes that too. SQL Server authentication credentials are then sent in cleartext each time the receiver opens a connection. Only use this on a trusted network.

`encrypt=disable` can only be set through `datasource`, which can't be combined with `server`, `port`, `username` or `password`. If you use those options, move the connection details into the connection string instead, for example:

```
receivers:
  sqlserver:
```

```
    datasource:
"sqlserver://${env:SQL_USER}:${env:SQL_PASSWORD}@<host>:1433?
encrypt=disable"
```

Special characters in the username or password must be URL-encoded in this form.

Scrapes appear to succeed but collect nothing on a failover cluster instance

On a SQL Server failover cluster instance (FCI), the instance listens on the cluster IP address rather than on the loopback address. Pointing the receiver at `localhost` therefore fails before any TLS handshake starts, with an error like:

```
dial tcp 127.0.0.1:1433: connectex: No connection could be made because
the target machine actively refused it.
```

This is easy to miss: after applying the TLS workaround above, the TLS errors stop, which looks like success while every scrape is actually failing. Point the receiver at the FCI's network name or cluster IP address instead.